Check for updates

IET Circuits, Devices & Systems

Research Article

Adjacency criticality: a simple yet effective metric for statistical timing yield optimisation of digital integrated circuits

ISSN 1751-858X
Received on 24th December 2018
Revised 3rd April 2019
Accepted on 7th May 2019
E-First on 22nd October 2019
doi: 10.1049/iet-cds.2018.5616
www.ietdl.org

Seyed Milad Ebrahimipour[1], Behnam Ghavami[1] ✉, Mohsen Raji[2]
[1]*Department of Computer Engineering, Shahid Bahonar University of Kerman, Kerman, Iran*
[2]*School of Electrical & Computer Engineering, Shiraz University, Shiraz, Iran*
✉ *E-mail: ghavami@uk.ac.ir*

Abstract: As CMOS devices become smaller, process variations-induced uncertainty imposes a large spread in the circuit timing and therefore, it becomes one of the main issues for circuit yield. To analyse/optimise the timing of the circuit under process variation effects, statistical analysis/optimisation techniques are more suitable than the traditional static analysis/optimisation counterparts. Statistical gate sizing is an effective technique that is widely used to guide the timing yield improvement of digital circuits. Gate criticality, defined as the probability that a gate lies on a critical path, forms the basis for many of the existing statistical gate sizing techniques. Here, the authors introduce adjacency criticality to address the drawbacks of the conventional definition of gate criticality. It is defined as the probability of manufacturing a chip in which the gate lies on the critical path due to process variation considering the effect of the gates in its fan-out cone. Furthermore, the authors present the levelised Adjacency Criticality metric which provides a trade-off between the runtime of the criticality metric and accuracy of the Adjacency Criticality metric. In order to show the efficacy of the proposed metric, an adjacency criticality-based statistical gate sizing method is presented for improving timing yield of the circuit.

1 Introduction

In order to gain more advances in different aspects such as performance and power consumption, integrated circuits manufacturing continuously has followed a trend of transistor dimensions shrinking [1]. As technology continues to scale down, the difficulty of precise fabrication process increases. This causes fabrication process-induced parameter variations (briefly process variations) such as changes in gate oxide thickness and threshold voltage which significantly affect the circuit timing and performance [2]. Process variations may change the circuit frequency up to 30% [3]. Thus, the gap between the designed layout and what is really fabricated on silicon is widening significantly. Handling the process variations effects at design time has the advantage of taking the right decisions early in the design flow which in turn, significantly reduces the time-to-market of the design. As a result, design-time compensating of process variations impacts is an inevitable requirement for modern digital circuits.

Deterministic timing analysis has been widely used to handle variability effects in very large-scale integrated (VLSI) circuit designs [4, 5]. However, this approach loses accuracy in analysing circuits under considerable process variations in nanoscale era. This is due to the fact that it estimates nominal, best-case, or worst-case delay of a circuit; best-case (worst-case) analysis significantly underestimates (overestimates) the process variations effects, and analysing circuits at nominal values of various circuit parameters completely ignores these variations [4, 5]. As a promising alternative, statistical analysis which is based on estimating the distribution performance of the circuit metrics using probability theory and statistics has shown more accurate results in variability-aware performance evaluation of digital circuits. Statistical static timing analysis (SSTA) is a widely used method for timing analysis in digital circuits in the presence of process variations. There are many SSTA methods that trade-off between time complexity and accuracy [6–11]. Statistical timing optimisation methods take advantages of SSTAs as the basis in order to improve the performance/yield of a digital circuit considering the impacts of process variations [12]. Hence, the efficacy of the statistical timing optimisation method is directly affected by the quality of the SSTAs as well as the information provided by the circuit analyser to be applied in the optimisation method [13].

In order to obtain specialised information for steering statistical optimisation, characteristic metrics have been introduced [6, 13–35]. Yield gradient (YG) which is an effective sensitivity metric for guiding the statistical timing optimisation flow is defined as the change in the timing yield (TY) of the circuit for small changes in the size of the gates [19, 24]. To assess the change in TY of the circuit, an SSTA is performed to obtain the change in the circuit delay due to a change in the size of the gates; i.e. YG is evaluated numerically. In [24], using a YG computation method, a gate sizing algorithm for parametric yield maximisation is introduced. Although YG is an accurate metric, YG-based optimisation algorithms are intrinsically iterative, and in each iteration, the YG of a large number of gates needs to be evaluated in order to designate the best candidate for resizing. Another metric, TY slack (YS) is introduced in [13]. The YS represents the extra TY available in a gate or a circuit. However, in order to compute YS of each gate, a large number of gates needs to be evaluated which is time-consuming for even mid-scale digital circuits.

In [20, 22], an indicator named as statistical timing criticality (STC) is introduced that represents the probability that the gate lies on the critical path considering process variations. STC is used for TY optimisation by improving the delay of the gates which have high STC values. In [25], the criticality of a path is expressed as the sensitivity of the mean of the normally distributed random variable of circuit delay to the mean of the path delay. In [6], a fast method to compute the criticality of the path and the node/arc is presented. This method is valid under the assumption that the paths are independent. However, due to correlated parameters and path reconvergence, it is not sufficiently accurate to be applied for TY optimisation of large and complicated circuits. Zhan *et al.* [26] compute the path criticality based on max operations and Monte–Carlo simulations. In this method, using a pre-selected path set, paths are picked out for criticality computation. However, this method suffers from a serious drawback, i.e. it is not clear how to determine the pre-selected path set (like path-based SSTA). Xiong *et al.* [27] introduced a cut-set-based method to compute criticality. After identifying all the cut set of the timing graph, the node/edge criticality is defined as the probability of the node/edge slack (the

© The Institution of Engineering and Technology 2019

maximum delay of all paths going through that node/edge) being greater than or equal to the complementary node/edge slack, i.e. the maximum delay of all paths which are not going through that node/edge. In order to overcome the error induced by max operator in criticality computation, a criticality computation method using unary and conditional operator is introduced in [28]. In [22, 29], an incremental criticality computation using circuit delay is introduced. In order to reduce the power consumption of the circuit, [31, 32] propose a dual supply voltage (DSV) technique in which a lower supply voltage is applied to gates such that the performance of the circuit maintains at the same time. To this end, these methods first compute the criticality of each gate. Then, DSV is applied to a group of the gates with the lowest criticality. At the presence of false paths, i.e. paths through which signals cannot propagate through it, finding the true critical paths is so difficult. If the false paths are the critical paths of the circuit, analysis of the circuit using SSTA would be too optimistic. Furthermore, criticality computation of the circuit would be inaccurate. Sreenath and Ramesh [33] propose a gate sizing method in which they first identify the false path of the circuit. Then, a group of gates selected from the true critical paths according to their criticality and gate sizing is applied to them.

In order to lessen the impact of process variation, post-silicon clock tuning elements are widely used in high-performance circuits. These elements can be configured through the scan chain so that clock skews to these flip-flops can be adjusted after manufacturing. Moreover, since these elements allow the path delays to compensate each other across flip-flops, critical paths in such circuits may span more than one stage which makes the criticality computation of such a circuit so challenging. In [34], a SSTA method is introduced to compute the performance of the circuit under post-silicon clock tuning elements. Furthermore, they propose a criticality computation method in these circuits so that an analysing-optimisation flow becomes possible.

In order to optimise the delay of the circuit, critical paths or nodes are the first targets in applying the circuit optimisation technique. Thus, it is important to select the critical gates correctly. However, all of the previous optimisation methods using criticality are suffering from a serious drawback. They did not consider the effect of the gates in the fan-out cone of each gate. Since optimising the delay of gate g will affect the arrival time (AT) of the gates in its fan-out cone, and as the criticality of the gates in the fan-out cone of a gate become higher, the effect of optimising that gate on the circuit delay become higher, neglecting the effect of the fan-out gates will give a more pessimistic (optimistic) criticality which slows down the convergence rate of the optimisation algorithm and result in high area overhead.

In this paper, we propose a novel optimisation metric called adjacency criticality (AC). It is defined as the probability of manufacturing a chip in which the gate lies on the critical path, due to process variation considering the effect of gates in its fan-out cone. In order to show the efficacy of the proposed metric, an AC-based statistical gate sizing method is presented for improving TY of the circuit. Experimental results on EPFL and ISCAS benchmark circuits show that the proposed metric leads to faster convergence of the optimisation algorithm and lower area overhead compared to the criticality metric.

The rest of this paper is organised as follows. Section2 provides an overview of the gate-level delay model and the SSTA used in this paper. In Section 3, we motivate the AC metric and the proposed algorithm to calculate AC of each gate is presented. In Section 4, the proposed gate sizing method to improve the TY of the circuit is introduced. Section5 reports the experimental results. In the end, the paper is concluded in Section 6.

2 Preliminaries: SSTA

Before explaining about SSTA, there are some parameters needed to be introduced:

- Required AT (RAT): 'the time within which the data is required to arrive at some internal node of the circuit'[5].
- Arrival Time (AT):'the time in which the data arrives at the internal node'[5]. It incorporates all the net and logic delays between the reference input point and the destination node.

In order to compute the delay distribution of the circuit, timing graph of the circuit is created. The timing graph is a directed acyclic graph $G(V, E)$ composed of nodes (V) and edges (E). The nodes represent different input pins of circuit gates. They are connected using edges that model signal propagation from one node to another with an associated delay demonstrating either gate or wire delay. Considering PV effects, this delay is a random variable with a specific probability density function (PDF). The timing graph is augmented by a single source node and a single sink node. The source node is a virtual node which is connected to all primary inputs via virtual edges and the virtual sink node is a virtual node which is connected to all primary outputs or timing test nodes with virtual edges.

Given the circuit timing graph and the statistical gate delay model as inputs, SSTA computes the statistical distribution of the circuit delay with linear runtime complexity [11]. In this paper, we apply the widely used canonical first-order delay model [6] with a relatively low complexity. The delay of a gate under PV effects considering spatial correlation can be stated as follows [33]:

$$D_{\text{gate}_i} = d_0 + \sum_{i=1}^{n} d_i \Delta X_i + d_{n+1} \Delta X_{\text{rand}} \tag{1}$$

where ΔX_i represents n-independent random variables used to express the spatially correlated device-parameter variations, ΔX_{rand} represents the residual-independent variation, d_i and d_{n+1} represent the sensitivity of the delay to each of the random variables, and d_0 is the mean value of D_{gate_i}.

Generally, SSTA consists of two repeating steps:

Step(1) Arrival Time Propagation: in this step, the AT propagates from the gate input to the gate output.
Step(2) Merging Arrival Times: this step merges multiple ATs converging them into the gate output from different gate inputs by calculating the maximum of the ATs.

2.1 Arrival time propagation

In this step, the gate delay is added to the AT. If $\text{AT}_{\text{gate}_i}$ and D_{gate_i}, respectively, represent the AT and delay of a gate in the canonical form, we have:

$$\text{AT}_{\text{gate}_i} = a_0 + \sum_{i=1}^{n} a_i \Delta X_i + a_{n+1} \Delta X_{\text{rand}} \tag{2}$$

$$D_{\text{gate}_i} = d_0 + \sum_{i=1}^{n} d_i \Delta X_i + d_{n+1} \Delta X_{\text{rand}} \tag{3}$$

The statistical sum of $\text{AT}_{\text{gate}_i}$ and D_{gate_i} can be shown in the canonical form of expression as follows:

$$\begin{aligned} \text{Sum} &= s_0 + \sum_{i=1}^{n} s_i \Delta X_i + s_{n+1} \Delta X_{\text{rand}} \\ s_0 &= a_0 + d_0 \\ s_i &= a_i + d_i \\ s_{n+1} &= \sqrt{a_{n+1}^2 + d_{n+1}^2} \end{aligned} \tag{4}$$

2.2 Merging arrival times

In this step, we compute the maximum AT of inputs of a gate. Due to non-linearity of the statistical maximum operation, the maximum of two parameters in canonical form cannot be expressed in the canonical form. In [5], an algorithm is introduced for computing a statistical approximation of the maximum of two

© The Institution of Engineering and Technology 2019

17518598, 2019, 7, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-cds.2018.5616 by Thapar University, Wiley Online Library on [23/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ATs: AT_1 and AT_2. First, it computes variance and covariance of A and B as:

$$\sigma^2_{AT_1} = \sum_{i=1}^{n+1} a_i^2 \sigma^2_{AT_2} = \sum_{i=1}^{n+1} b_i^2 \rho \sigma_{AT_1} \sigma_{AT_2} = \sum_{i=1}^{n} a_i b_i \tag{5}$$

where, $\sigma^2_{AT_1}$and $\sigma^2_{AT_2}$ are respectively the variances of AT_1 and AT_2, and ρ denotes the correlation coefficient of AT_1 and AT_2. Then, the tightness probability (T_A), which is defined as the probability of that AT AT_1, is larger than AT_2, is computed as:

$$T_A = P(AT_1 > AT_2) = \phi\left(\frac{a_0 - b_0}{\theta}\right) \tag{6}$$

in which:

$$\phi(y) = \int_{-\infty}^{y} \varnothing(x)\,\mathrm{d}x \tag{7}$$

$$\varnothing(x) = \frac{1}{\sqrt{2\pi}}\,\mathrm{e}^{-(x^2/2)} \tag{8}$$

$$\theta = \sqrt{\sigma_A^2 + \sigma_B^2 - 2\rho\sigma_A\sigma_B} \tag{9}$$

where $\phi(y)$ and $\varnothing(x)$ are standard Gaussian cumulative distribution function (CDF) and standard PDF, respectively. Later, the mean and variance of $Y = \max(A, B)$ are computed as follows:

$$Y = \max(AT_1 \cdot AT_2) = y_0 + \sum_{i=1}^{n} y_i \Delta V_{\mathrm{th}(i)} + y_{n+1}\Delta V_{\mathrm{th_rand}} \tag{10}$$

$$y_0 = a_0 T_A + b_0(1 - T_A)\theta\varnothing\left(\frac{a_0 - b_0}{\theta}\right) \tag{11}$$

$$\begin{aligned}\sigma_Y^2 = &\left(a_0^2 + \sigma^2_{AT_1}\right) + \left(b_0^2 + \sigma^2_{AT_2}\right)(1 - T_A) \\ &+ (a_0 + b_0)\theta\,\varnothing\left(\frac{a_0 - b_0}{\theta}\right) - y_0^2\end{aligned} \tag{12}$$

Finally, the sensitivity coefficients are calculated by:

$$y_i = a_i T_A + b_i(1 - T_A) \tag{13}$$

$$y_{n+1} = \sqrt{\sigma_Y^2 - \sum_{i=1}^{n} y_i^2} \tag{14}$$

3 Adjacency criticality

Due to process variation effects, path delays are no longer deterministic values, but are described by random variable with a specific PDF. This implies that each of the paths can be critical with some probability, and none of the paths can be critical with absolute certainty; i.e. we have a set of critical paths instead of only a critical path. Since the critical path set changes from die to die due to process variations effects, computing criticality is a crucial step in TY optimisation. As defined in [21], '*Criticality of a path is the probability of manufacturing a chip in which the path of interest is critical*'. '*Criticality of a gate is defined as the probability of manufacturing a chip in which the gate lies on the critical path, due to process variation*'[21]. In other words, the criticality of a gate is the criticality of the set of all paths going through that gate. Using criticality, critical paths or gates become the key nominees to apply the circuit optimisation techniques [27]. Thus, in order to provide noteworthy information for guiding statistical optimisation, it is important to accurately compute the criticality of each gate. Since optimising the delay of gate g will affect the AT of the gates in its fan-out cone, neglecting the effect of the fan-out gates will lead to an inaccurate criticality which slows down the convergence rate of the optimisation algorithm. For

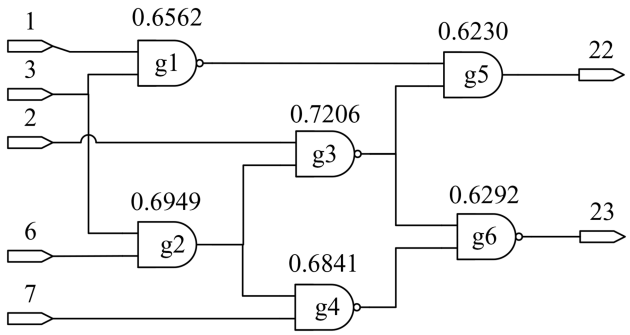

Fig. 1 *Combinational circuit with associated criticality*

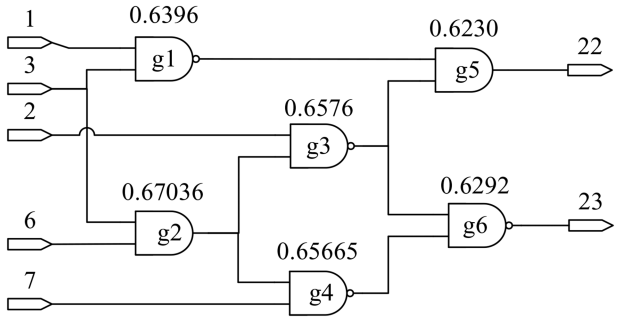

Fig. 2 *Combinational circuit with associated adjacency criticality*

example, consider the combinational circuit shown in Fig. 1. Each gate of the circuit is annotated with its criticality. The criticalities in this example are computed using the method proposed in [28]. As it can be seen, gate $g3$ has the largest criticality and it will be the first candidate of the optimisation algorithm. However, if we consider the effect of the gates in the fan-out cone of each gate, gate $g2$ will be chosen for optimisation because $g3$ and $g4$ which are located in the fan-out cone of $g2$ have the largest criticalities and optimising $g2$ will affect the delay of the paths going through $g3$ and $g4$. As a result, it leads to a faster optimisation process.

All of the optimisation methods using criticality are suffering from a serious drawback; i.e. they did not consider the effect of the gates in the fan-out cone of each gate. As the criticality of the gates in the fan-out cone of g becomes higher, the effect of optimising g on the circuit delay increases. Therefore, we need to select a gate that not only has a high criticality but also the criticality of the gates in its fan-out cone is high.

Using this intuition, we define AC concept. The AC of a gate is defined as the probability of that the gate lies on the critical path considering the effect of gates in its fan-out cone. In order to compute AC of each gate, the average of the criticality of the gates in the fan-out cone of that gate is calculated. The AC of gate_i is expressed as:

$$\mathrm{AC}_{\mathrm{gate}_i} = \frac{c_i + \sum_{j=1}^{n} c_j}{1 + n} \tag{15}$$

where c_i is the criticality of gate_i and n is the number of gates in the fan-out cone of gate_i. Fig. 2 shows the AC of each gate computed using (15) for the given combinational circuit. As expected, using the proposed equation, gate $g2$ has the largest AC and will be the first candidate for the optimisation algorithm.

Fig. 3 (See Algorithm 1) aims to compute $\mathrm{AC}_{\mathrm{gate}_i}$ for each gate of the circuit using (15). The circuit and the SSTA results are the inputs to this algorithm. At first, the criticality of each gate is computed (line 7, 8). Then, each gate of the circuit is traversed (line 10). During each traversing, the average criticality of the gates in fan-out cone of each gate is computed (line 11–21). If the gate is an output gate (a gate with no predecessor), the average criticality of this gate is equal to its criticality (line 11–15). The average criticalities of other gates are computed by calculating the average of the criticality of their predecessors (line 16–19). Finally,

© The Institution of Engineering and Technology 2019

17518598, 2019, 7, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-cds.2018.5616 by Thapar University, Wiley Online Library on [23/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

1. Inputs : Circuit, SSTA results
2. Output : Adjacency Criticality of each gate

---

3. c_i : $Criticality$ of $gate_i$
4. AC_i : $Adjacency_Criticality$ of $gate_i$
5. $AC_{fan_out_i}$: $Adjacency_Criticality$ of $fan_{out}cone$ of $gate_i$
6. $n_{fan_out_i}$: $number$ of $gates$ in $fan_{out}cone$ of $gate_i$
7. **for each** $gate_i$ **in** *Circuit* **do**
8. Compute c_i of $gate_i$
9. **end for**
10. **for each** $gate_i$ **in** *Circuit* **do**
11. **if** ($gate_i$ *has no predecessor*)
12. $AC_{fan_out_i} = c_i$
13. $n_{fan_out_i} = 0$
14. **Continue**
15. **end if**
16. **for each** $gate_u$ **in** *fan-out cone* **of** $gate_i$ **do**
17. $AC_{fan_out_i} = AC_{fan_out_i} + c_u$
18. $n_{fan_out_i} = n_{fan_out_i} + 1$
19. **end for**
20. $AC_i = \frac{c_i + AC_{fanout_i}}{n_{fan_out_i} + 1}$
21. **end for**

Fig. 3 *Algorithm 1: Adjacency Criticality Computation*

1. Inputs : Circuit, SSTA results
2. Output : Adjacency Criticality of each gate

---

3. c_i : $Criticality$ of $gate_i$
4. AC_i : $Adjacency_Criticality$ of $gate_i$
5. $AC_{fan_out_i}$: $Adjacency_Criticality$ of $fan_{out}cone$ of $gate_i$
6. $n_{fan_out_i}$: $number$ of $gates$ in $fan_{out}cone$ of $gate_i$
7. **for each** $gate_i$ **in** Circuit **do**
8. Compute c_i of $gate_i$
9. **end for**
10. **for each** $gate_i$ **in** $topological\ order$ **do**
11. **for** j **in** 1 **to** *number of levels* **do**
12. **for each** $gate_k$ **in** $level_j$ **do**
13. $AC_{fan_out_i} = AC_{fan_out_i} + c_k$
$n_{fan_out_i} = n_{fan_out_i} + 1$
14. **end for**
15. **end for**
16. $AC_i = \frac{c_i + AC_{fanout_i}}{n_{fan_out_i} + 1}$
17. **end for**

Fig. 4 *Algorithm 2: Levelised Adjacency Criticality Computation*

using the average criticality of fan-out cone of each gate, the AC of each gate is computed (line 20).

Computing AC of each gate using (15) is time-consuming because it is required to traverse the fan-out cone of each gate to find its AC. As expressed in [29–31], the impact of changes in the delay of a gate (by a gate resizing) decreases (approximately geometrically) with increasing fan-out level. Therefore, in order to compute the AC of each gate, it is only required to consider the effect of the changes in the gate delay in a few levels of its fan-out cone. The question is, for a reasonable accuracy, how many levels of fan-out are required to evaluate. Choosing k (number of levels) is a trade-off between runtime and accuracy. As a result, The AC of gate_i can be expressed as:

$$\text{AC}_{\text{gate}_i} = \frac{c_i + \sum_{j=1}^{\#\text{levels}} \sum_{l=1}^{k} c_{j.l}}{1 + \sum_{j=1}^{\#\text{levels}} k_j} \tag{16}$$

1. Inputs : Circuit, SSTA results, Adjacency Criticality of each gate
2. Output : Optimized Circuit

---

3. n : $number\ of\ gate\ chosen\ for\ gate\ sizing$
4. a_t_i : $Average_Criticality$ of $gate_i$
5. **for each** $gate_i$ **in** $Circuit$ **do**
6. $Compute\ a_t_i\ of\ gate_i$
7. **end for**
8. $Rank\ gates\ based\ on\ a_t_i$
9. $Select\ first\ n\ gates\ for\ gate\ sizing$
10. **for each** $gate_j$ **in** n
11. $increase\ the\ size\ of\ gate_j\ by\ 1$
12. **end for**
13. $Compute\ Timing\ Yield\ of\ the\ Circuit$
14. **if** ($Timing\ Yield\ Objective\ achieved$)
15. **go to** $line\ 20$
16. $Compute\ Area\ of\ the\ Optimized\ Circuit$
17. **if** ($Area\ Constraint\ violated$)
18. **go to** $line\ 20$
19. **go to** $line\ 4$
20. $Compute\ Area\ of\ the\ Optimized\ Circuit$

Fig. 5 *Algorithm 3. Gate Sizing Algorithm*

where $c_{j.l}$ and k_j represent criticality of lth gate and number of gates in level j, respectively.

The overall algorithm to compute AC considering some levels of fan-out cone is presented in Fig. 4 (See Algorithm 2). At first, the criticality of each gate is computed (line 7, 8). Then, each gate is visited in the topological order (line 10). During each traversing, the average criticality of the gates in fan-out cone of each gate is computed (line 11–17). The average criticality of each gate is computed by calculating the average of the criticality of the gates in some levels, i.e. one or two levels, of its fan-out cone (line 11–15). Finally, using the average criticality of fan-out cone of each gate, the AC of each gate is computed (line 16).

In order to compute AC, we need to compute the criticality of each gate. Without loss of generality, the incremental criticality computation method introduced in [21, 28] is used to find the criticality of each gate. However, any of the similar methods can be adapted to the proposed algorithm.

Incremental timing analysis is a corner-stone of optimisation and physical synthesis, in which the timing quantities can be updated incrementally after one or more changes are made to the circuit. In order to compute the criticality of node $v \in V$ incrementally, all the paths from the source node to the sink node are divided into two disjoint sets. The first set comprises the paths going through node v, while the second one comprises the paths which are not passing through node v. The maximum delay of paths going through v is node slack A which is expressed as:

$$A = \text{AT}_v - \text{RAT}_v \tag{17}$$

where AT_v and RAT_v, respectively, represent the AT and required AT of node v.

The maximum delay of paths not going through v is called *complement node slack B*. The node criticality is the probability of *node slack A* being greater than or equal to the *complement node slack B*, i.e.:

$$t = P(A \geq B) \tag{18}$$

Criticality of a node (gate) can be computed as [21, 28]:

$$\begin{aligned} t &= P(A \geq B) = P(A \geq \text{A} \ \& \ A \geq B) = P(A \geq \max(\text{AB})) \\ &= P(A \geq C), \end{aligned} \tag{19}$$

where A is the node slack, B is the complement node slack, and $C = \max(A, B)$ is the chip slack, which is the maximum of the delay of all paths from the source to the sink or simply AT of the sink

© The Institution of Engineering and Technology 2019

17518598, 2019, 7, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-cds.2018.5616 by Thapar University, Wiley Online Library on [23/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 1 Comparison of delay distribution parameters for the circuits after optimising with AC, levelised AC, and criticality metrics under 15 iterations constraint

Circuit		Criticality		Levelised adjacency criticality						Adjacency criticality	
				$K = 1$		$K = 2$		$K = 3$			
Bench	#Gates (#PI, #PO)	$\%\mu$	$\%\sigma^2$	$\%\mu$	$\%\sigma^2$	$\%\mu$	$\%\sigma^2$	$\%\mu$	$\%\sigma^2$	$\%\mu$	$\%\sigma^2$
C499	575 (41,32)	1.69	2.17	9.03	15.75	9.69	15.94	13.02	17.87	13.60	21.71
C1908	524 (33,25)	11.22	18.28	14.02	21.06	21.02	28.52	23.10	40.67	23.86	33.23
C2670	834 (233,140)	5.51	9.37	8.28	14.97	14.52	25.11	16.68	33.84	19.81	33.16
C3540	1088 (50,22)	8.15	9.22	9.77	11.92	13.73	14.57	18.97	21.81	21.14	26.56
C5315	1666 (178,123)	9.41	17.50	12.18	23.25	18.27	32.79	29.04	45.02	36.24	59.44
C6288	2416 (32,32)	4.75	11.26	6.66	15.20	9.42	18.11	10.51	21.25	11.83	24.85
C7552	3513 (207,108)	5.83	9.15	10.49	19.19	17.37	30.44	29.22	50.33	33.03	54.98
max	5824 (512,130)	18.78	29.12	20.54	34.98	22.36	37.48	24.16	41.12	26.32	45.15
sin	11,031 (24,25)	5.45	10.47	8.49	15.17	13.74	24.91	16.98	33.48	20.08	33.76
arbiter	23,747 (256,129)	5.68	11.12	9.08	15.83	14.36	25.31	17.84	35.39	19.93	36.42
voter	31,622 (1001, 1)	20.31	41.63	25.83	49.95	31.13	57.70	36.23	61.37	38.56	66.08
square	39,558 (64,128)	8.58	11.20	12.81	15.78	16.53	24.17	19.78	36.41	23.89	42.07
sqrt	54,215 (128, 64)	8.97	18.74	9.25	19.27	10.16	20.97	13.01	26.09	15.71	30.61
multiplier	55,910 (128, 128)	21.89	37.79	24.20	41.88	26.63	46.05	28.48	49.19	30.60	59.93
average		**9.73**	**16.93**	**12.90**	**22.44**	**15.93**	**28.72**	**21.22**	**36.71**	**23.9**	**40.57**

node. In other words, the criticality of node v is the probability that its node slack dominates the chip slack. Thus, we have [21]:

$$t = P(\mathrm{AT}_v - \mathrm{RAT}_v \geq \mathrm{AT}_{\mathrm{sink}}) \tag{20}$$

where $\mathrm{AT}_{\mathrm{sink}}$ represents AT of the sink node.

Therefore, to update node criticality t, we only need to update AT_v, RAT_v and $\mathrm{AT}_{\mathrm{sink}}$. In order to do this efficiently and incrementally, it is just required to run the SSTA for the fan-in cone and fan-out cone of node v. In fact, after incrementally updating these quantities, computing gate criticality using (20) can be done in a constant time [21].

4 Timing yield optimisation using adjacency criticality

In this section, we introduce a statistical gate sizing algorithm for TY optimisation using AC. Gate sizing is one of the simplest yet effective methods for timing optimisation. The TY is the number of fabricated chips whose delay is at or below a specified value called the timing constraint. In other words, '*the timing yield of the circuit denotes the probability that the circuit slack is non-negative*'[36]. This probability can be computed by integrating the circuit slack PDF from 0 to ∞. Given the circuit slack as a Gaussian-distributed random variable with mean μ_S and standard deviation σ_S, the TYof the circuit which we try to improve is given by [36]:

$$\mathrm{TY} = \frac{1}{\sqrt{2\pi}\sigma_S}\int_0^{\infty} \mathrm{e}^{-((x-\mu_S)^2/2\sigma_S^2)}\mathrm{d}x \equiv \phi\left(\frac{\mu_S}{\sigma_S}\right) \tag{21}$$

where $\phi(\cdot)$ represents the standard normal cumulative distribution function.

It is worthy to note that, as the yield of the circuit is improved, the probability of having a circuit with a delay smaller than or equal to the timing constraint increases.

Fig. 5 (See Algorithm 3) shows the overall gate sizing algorithm.

After the AC of each gate in the circuit is computed, since the effect of updating different gates are different and interdependent, it is important to select a group of the most effective gates in each iteration of the optimisation algorithm. As a result, the gates are ranked according to their AC and a group of the highest-ranking gates is chosen for optimisation. After each iteration of optimisation, SSTA is carried out to obtain the timing of the circuit. It is notable that, in the SSTA, only fan-in gates of the n modified gates and those on their fan-out cones are required to be re-examined. This iterative sizing procedure terminates when the timing constraint is satisfied (i.e. the TY objective is achieved) or the maximum area constraint is violated.

5 Experimental results

The proposed algorithm of AC computation is implemented in C++ and run on a Microsoft Windows machine with an Intel Core i7 (2.4 GHz) processor and an 8 GB of RAM circuit. All of the gates in the circuit are assigned with a base size, i.e. the W/L ratio of PMOS in an inverter is set to 2 and W/L ratio for its NMOS is set to 1. In addition, for other gates (NAND, NOR etc.), we have sized the gates such that they have pull-up and pull-down networks with equal current strengths. In this paper, we assume process variations in effective channel length and threshold voltage. However, one could consider other source of variations; for example in the channel width and oxide thickness of the transistor. All of the random variables are set to the standard Gaussian random variable $N \sim (0, 1)$.

The experiments are divided into two parts. At first, we compare the AC concept with the criticality concept using various constraints. Then, in the second part, we investigate the efficiency of the proposed gate sizing optimisation algorithm using AC concept by applying the algorithm to some benchmark circuits.

5.1 Comparison of AC concept with criticality concept

In order to evaluate the efficiency of AC for guiding statistical optimisation, we compare the obtained results of improving the delay of the circuit using the gate sizing based on the AC and conventional definition of the gate criticality introduced in [21, 28]. The gate sizing algorithm applied for 15 iterations and in each iteration, 10% of the gates are picked out from the candidate gate set given their optimising priorities. The effectiveness of each metric is assessed based on the improvement achieved in the circuit delay. Table 1 shows the obtained results. In Table 1, the first seven rows are related to ISCAS'85 benchmark circuits and the other rows show some circuits from EPFL benchmark suit. In addition, columns named as '$\%\,\mu$' and '$\%\,\sigma^2$', respectively, represent the percentage of the mean and variance relative improvement provided by the gate sizing. As reported in this table, on average, the AC-based gate sizing improves the mean of the circuit by 23.9% which is >9.73% mean improvement achieved when using the criticality-based gate sizing. The AC-based gate sizing method also improves the variance of the circuit by 40.57% which is >16.93% variance improvement obtained by criticality-based gate sizing. The comparison demonstrates that AC is an effective metric for guiding statistical optimisation which leads to a better circuit delay (both in mean and variance) in comparison with the

© The Institution of Engineering and Technology 2019

17518598, 2019, 7, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-cds.2018.5616 by Thapar University, Wiley Online Library on [23/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

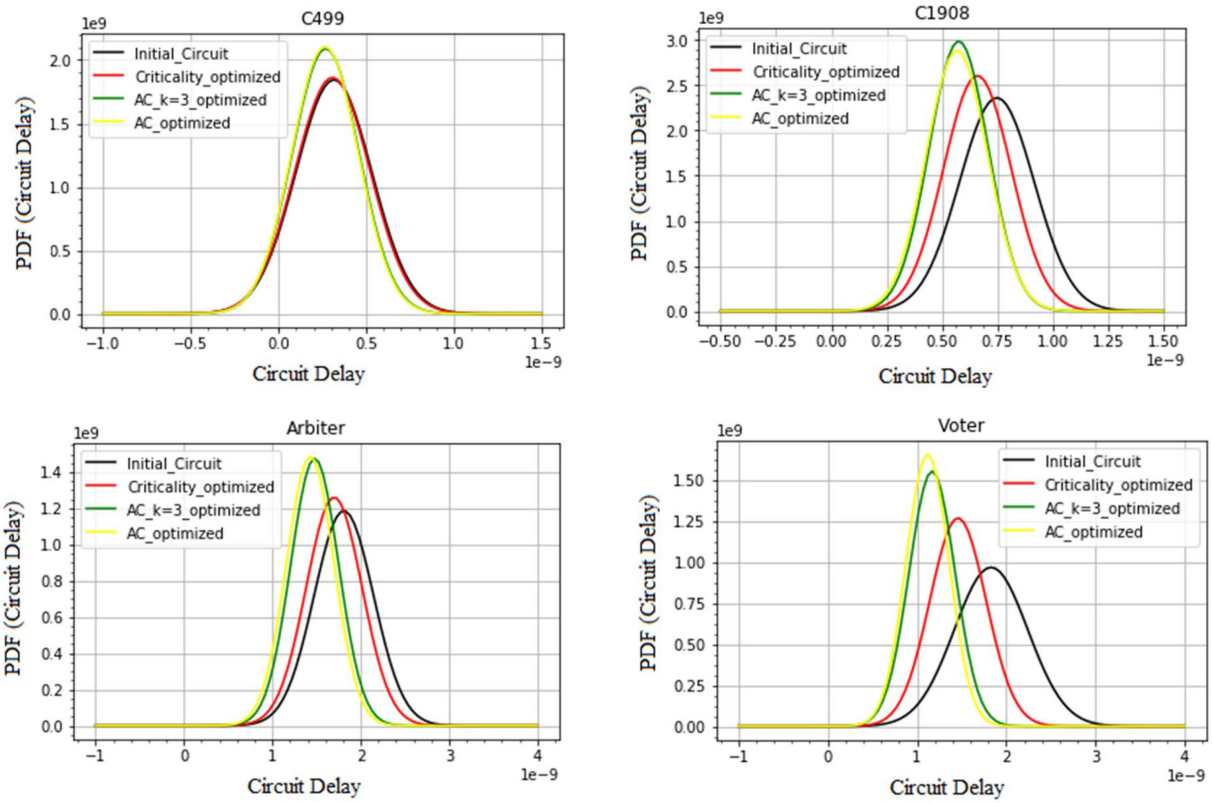

Fig. 6 *Comparison of the delay distribution of the circuit before and after optimisation using AC and criticality for C499, C1908, Arbiter, and Voter circuit*

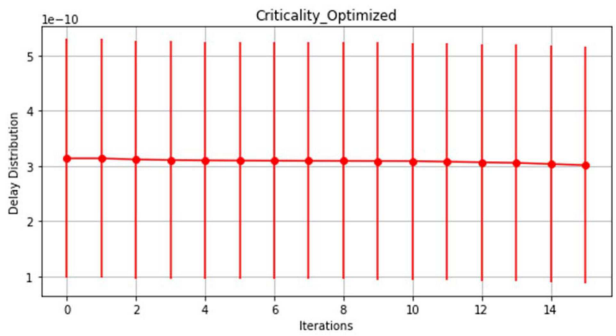

Fig. 7 *Delay distribution of C499 benchmark circuit after each iteration of optimisation using criticality*

conventional gate criticality metric. This table also shows the results for levelised AC under different k, i.e. number of fan-out levels. As it can be seen, for $k = 1$, the results are close to the criticality-based gate sizing results. However, for $k = 3$, the results are closer to the results obtained from the AC-based gate sizing as compared to $k = 1$ and $k = 2$.

Fig. 6 compares the delay distribution of the circuit before and after the optimisation procedure with AC, levelised AC with $k = 3$, and criticality metrics, after 15 rounds of optimisation, for C499, C1908, Arbiter, and Voter as a set of examples. As it can be seen, the mean and variance of the delay distribution of the circuits which are optimised with AC metric is smaller than the circuit optimised with the criticality concept. As a result, the TY of the circuit that is optimised with AC metric is more than the TY of the circuit optimised with the traditional criticality one. The results also show that the optimised delay distribution of the circuit using levelised criticality with $k = 3$ is very close to the delay distribution that is optimised with AC which shows the effectiveness of levelised AC metric.

Figs. 7 and 8 compare the mean and variance of the circuit delay after each iteration of optimisation procedure for C499 benchmark circuit using AC, levelised AC, and criticality metrics for 15 rounds of optimisation. As it can be seen, using AC concept, the mean and variance of the delay of the circuit are reduced the slope of this reduction (both in the mean and variance) is also more than that one in the figure shown for criticality-based optimisation method.

To show the efficiency of AC concept, the proposed AC-based optimisation algorithm is compared with the criticality-based optimisation method considering different area overhead constraints. The proposed method is applied for two different cases in which the area overhead is limited to 10 and 20% of the initial circuit area. Figs. 9 and 10 compare the obtained results for the circuit delay improvement before and after optimisation based on AC, levelised AC, and criticality metric. Here, the delay of the circuit is computed and reported as $(\mu + 3\sigma)$ of the circuit delay distribution. As it can be seen, in the same area constraint, the delay improvement of the circuit optimised with AC is greater than the delay of the circuit optimised with criticality; i.e. with a specific area overhead, AC-guided optimisation leads to a circuit with higher TY than criticality-based optimisation algorithm. The delay optimisation using levelised AC varies between AC and criticality concept. As the number of fan-out levels increases, the delay improvement will be closer to the AC delay improvement.

In order to investigate the runtime of the optimisation algorithm, we compare the runtime of the AC-guided optimisation algorithm with the algorithm based on the criticality metrics. Table 2 reports the obtained results. In this table, columns named as '% μ' and '% σ^2', respectively, represent the percentage of the mean and variance average relative improvement provided by the gate sizing, in each iteration. In addition, column 'RTPI' denotes the algorithm runtime per iteration, i.e. how long each iteration takes. As the results show, due to higher computation time required for AC calculation, the runtime of each iteration of AC-based optimisation algorithm is on average 3.85×more than the runtime of each iteration of criticality-based optimisation algorithm (723.11(s) compared to 187.87(s)). Despite this disadvantage, each iteration of AC-based gate sizing improves the mean of the circuit by 1.59% which is >0.64% mean improvement achieved in each iteration of the criticality-based gate sizing optimisation. Each iteration of the AC-based gate sizing method also improves the variance of circuit 2.54% which is >1.12% variance improvement obtained in each iteration of criticality-based gate sizing.

© The Institution of Engineering and Technology 2019

17518598, 2019, 7, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-cds.2018.5616 by Thapar University, Wiley Online Library on [23/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

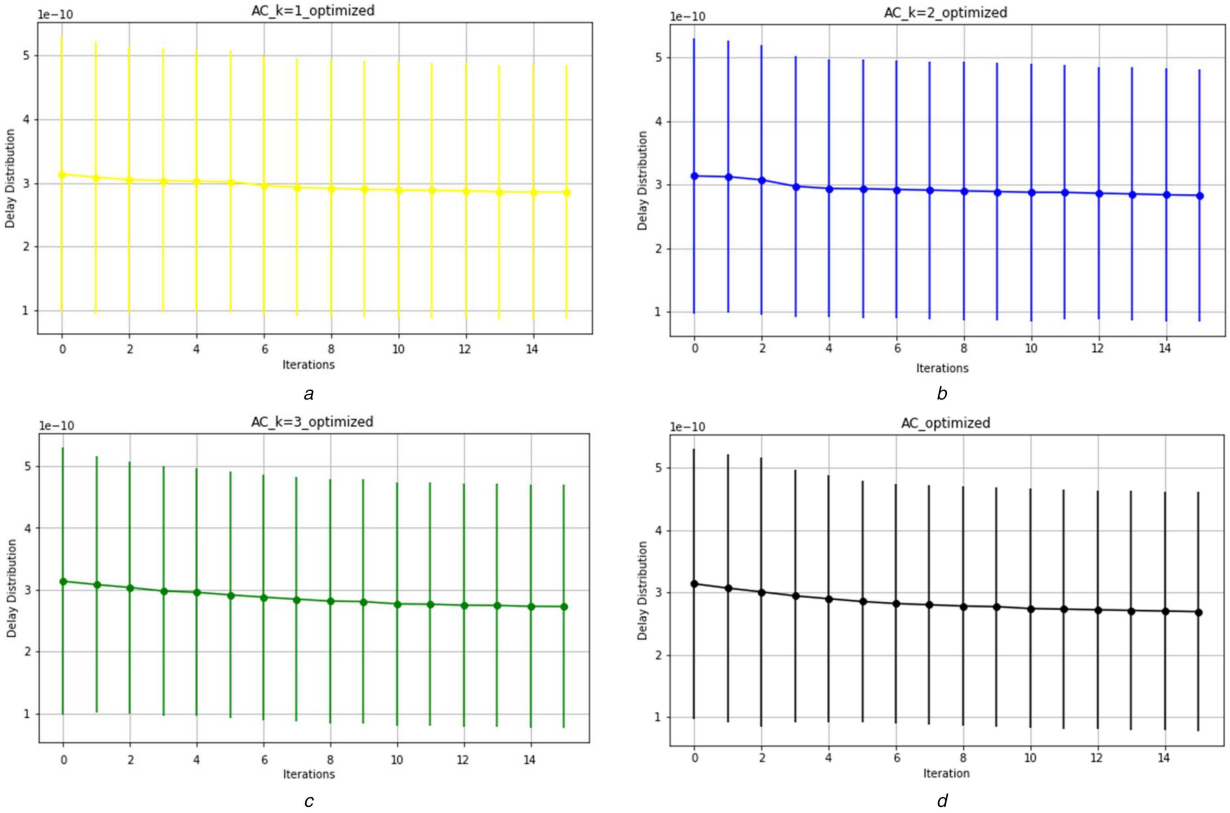

Fig. 8 *Comparison of delay distribution of C499 benchmark circuit after each iteration of optimisation using:* ***(a)*** levelised AC with $k = 1$, ***(b)*** levelised AC with $k = 2$, ***(c)*** levelised AC with $k = 3$, and ***(d)*** AC concept

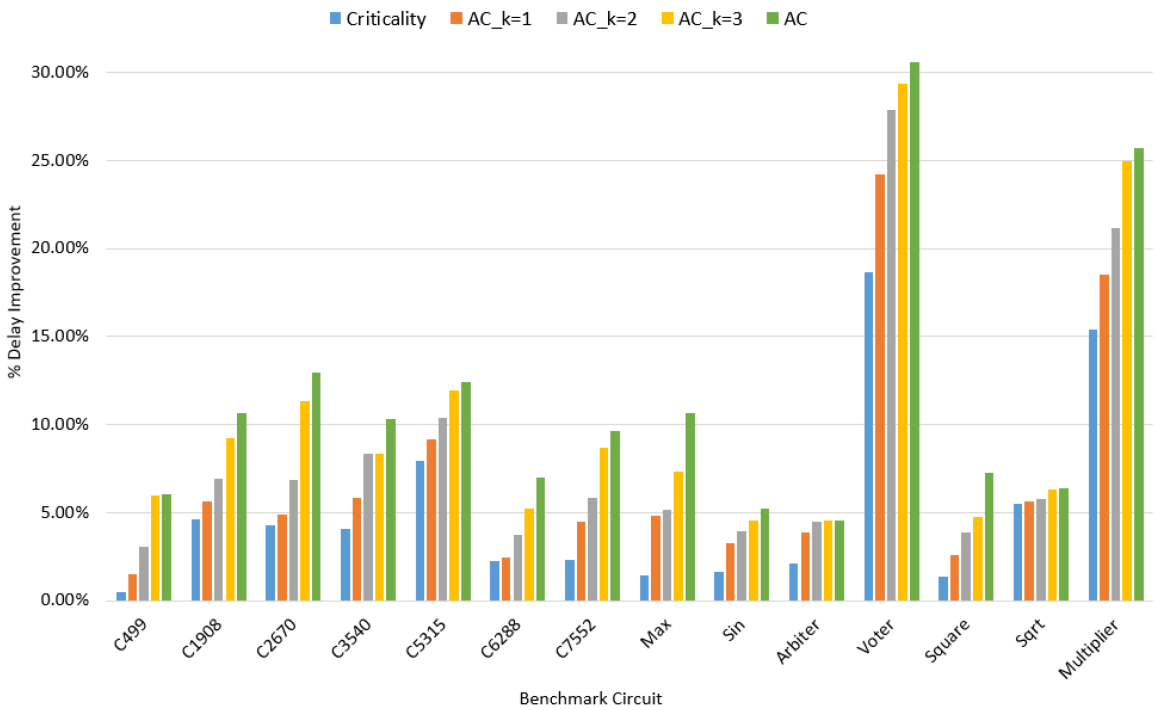

Fig. 9 *Comparison of circuit delay distribution after optimisation with AC, levelised AC, and criticality concept under 10% area overhead constraint*

To compensate the runtime of the AC-based gate sizing algorithm, one could use the levelised AC-based gate sizing algorithm. As reported in Table 2, for $k = 3$, the runtime of each iteration of the levelised AC-based gate sizing algorithm is about 406.12(s) which is 1.78× faster than the runtime of each iteration of the AC-based gate sizing algorithm. Furthermore, at each iteration, the levelised AC-based gate sizing algorithm improves the mean and variance of the circuit by 1.41% and 2.44%, respectively, which is close to the results obtained from the AC-based gate sizing algorithm.

It is notable that choosing k for the levelised AC is a trade-off between runtime and accuracy. As shown in Table 2, by increasing k, the runtime of the algorithm increases (the runtime of each iteration of gate sizing algorithm based-on the levelised AC metric are 277.86(s) for $k = 1$, 325.31(s) for $k = 2$, and 406.12(s) for $k = 3$) but meanwhile, the mean and variance improvement of the optimisation algorithm become closer to the obtained results from the AC-based gate sizing optimisation, i.e. the average relative mean and variance improvement of each iteration of the levelised AC-based gate sizing algorithm are, respectively, 1.41 and 2.44%,

© The Institution of Engineering and Technology 2019

17518598, 2019, 7, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-cds.2018.5616 by Thapar University, Wiley Online Library on [23/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

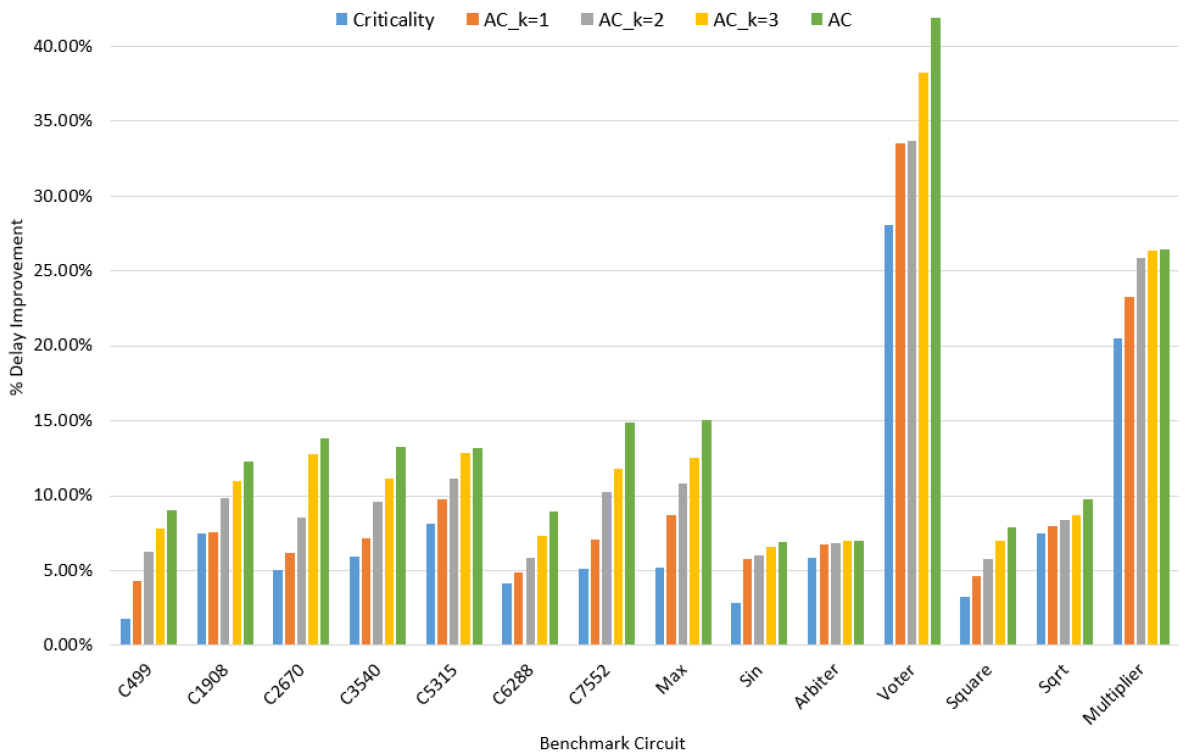

Fig. 10 *Comparison of $(\mu + 3\sigma)$ of circuit delay distribution after optimisation with AC, levelised AC, and criticality concept under 20% area overhead constraint*

Table 2 Comparison of the runtime and delay distribution parameters for the circuit after each round of gate sizing optimisation using AC, levelised AC, and criticality concept

	Criticality			Levelised adjacency criticality									Adjacency criticality		
				$K = 1$			$K = 2$			$K = 3$					
Bench	% μ	% σ^2	RTPI	% μ	% σ^2	RTPI	% μ	% σ^2	RTPI	% μ	% σ^2	RTPI	% μ	% σ^2	RTPI
C499	0.11	0.14	0.15	0.6	1.05	0.18	0.64	1.06	0.19	0.86	1.19	0.19	0.9	1.44	0.21
C1908	0.74	1.21	1.05	0.93	1.4	1.32	1.40	1.90	1.34	1.54	2.71	1.36	1.59	2.21	1.85
C2670	0.36	0.62	1.55	0.55	0.99	2.08	0.96	1.64	2.12	1.11	2.25	2.15	1.32	2.21	2.62
C3540	0.54	0.61	1.79	0.65	0.79	2.4	0.91	0.97	2.61	1.26	1.45	2.84	1.40	1.77	3.98
C5315	0.62	1.16	3.5	0.81	1.55	4.65	1.21	2.18	4.93	1.93	3	5.25	2.41	3.96	7.11
C6288	0.31	0.75	3.73	0.44	1.01	4.72	0.62	1.20	5.18	0.7	1.41	5.6	0.78	1.65	6.46
C7552	0.38	0.61	5.28	0.69	1.27	6.53	1.15	2.02	7.05	1.94	3.35	7.5	2.2	3.66	10.97
max	1.25	1.94	10.7	1.37	2.33	14.68	1.49	2.5	16.49	1.61	2.74	19.66	1.75	3.01	48.90
sin	0.36	0.7	24.28	0.57	1.01	38.44	0.92	1.66	49.83	1.13	2.23	61.26	1.34	2.25	191.33
arbiter	0.38	0.74	126.67	0.61	1.06	198.54	0.96	1.69	232.78	1.19	2.36	284.4	1.33	2.43	629.51
voter	1.35	2.77	298.5	1.72	3.33	412.03	2.07	3.84	461.36	2.41	4.09	543.09	2.57	4.4	1091.74
square	0.57	0.74	403.51	0.85	1.05	615.51	1.1	1.61	746.51	1.31	2.42	917.54	1.59	2.8	1573.86
sqrt	0.59	1.24	856.56	0.61	1.28	1274.35	0.67	1.39	1497.76	0.86	1.73	1877.56	1.04	2.04	3181.67
multiplier	1.45	2.51	892.89	1.61	2.79	1314.67	1.77	3.07	1526.12	1.89	3.27	1957.32	2.04	3.99	3373.26
average	**0.64**	**1.12**	**187.87**	**0.86**	**1.49**	**277.86**	**1.06**	**1.91**	**325.31**	**1.41**	**2.44**	**406.12**	**1.59**	**2.54**	**723.11**

which are higher than the average relative mean and variance improvement of gate sizing based on the levelised AC with $k = 1$ (0.86% mean improvement and 1.49% variance improvement) and 1.06% average relative mean improvement and 1.91% average relative variance improvement obtained from the gate sizing algorithm based on the levelised gate sizing algorithm with $k = 2$.

5.2 Yield optimisation

In order to evaluate the convergence rate of the proposed gate sizing technique, we conduct an experiment where we use a dynamic threshold in which the gates with AC greater than a fraction (0.95) of the maximum AC (AC_{max}) are chosen for optimisation, in every iteration. The algorithms stop when they achieve the target TY constraint which is 97% or the area overhead constraint which is 20% of initial circuit area is violated. Table 3 shows the obtained results in which the columns named as 'Iterations' and '%Yield' denote the number of iterations for convergence and the percentage of TY achieved by the sizing procedure, respectively. Column '%Area' presents the percentage of area overhead imposed by the optimisation algorithm. The obtained results show that the proposed gate sizing method using AC concept can achieve the target yield constraint with only 2.79 iterations and 10.02% area overhead, on average.

6 Conclusion

Diagnostic metrics are the cornerstone of optimisation to provide valuable information for guiding statistical optimisation algorithms. In this paper, we introduce a novel optimisation metric called AC. Then, using the proposed metric, a gate sizing-based optimisation algorithm is presented that aims to improve the TY of a given circuit. Extensive experimental results are conducted to investigate the efficacy of the proposed metric with respect to the

© The Institution of Engineering and Technology 2019

17518598, 2019, 7, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-cds.2018.5616 by Thapar University, Wiley Online Library on [23/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 3 Number of iterations needed for convergence of the algorithm under 97% target yield or 20% area overhead constraint

Bench	Adjacency criticality		
	Iterations	% Yield	% Area
C499	2	97	7.56
C1908	1	97	5.19
C2670	1	97	6.61
C3540	4	97	10.48
C5315	5	97	12.57
C6288	2	97	8.79
C7552	2	97	9.24
max	3	97	10.73
sin	2	97	11.61
arbiter	4	97	12.76
voter	2	97	8.91
square	5	97	13.59
sqrt	3	97	9.41
multiplier	3	97	12.86
average	**2.79**	**97**	**10.02**

TY improvement, area overhead, optimisation runtime, and convergence rate. Comparing to the conventional criticality metric, the obtained results show the superiority of the proposed AC metric when used as a diagnostic metric in a statistical optimisation algorithm in all aspects considered in the experiments.

As discussed in [29], a change in the size of a gate increases the load capacitance seen by the gates in its fan-in cone. Therefore, it increases their delay and output slew. So, as a future extension of our work, the effect of fan-in gates on the AC of a gate should be taken into account.

It is worthy to note that in this paper, we have assumed that the gates and circuit timing follow Gaussian distribution and thus, we use Gaussian distribution-based criticality computation method. However, the proposed AC metric is independent of the type of the distribution of random variables and one could use criticality computation methods presented in the [19, 21, 37] for computing criticality in digital circuits that does not follow the Gaussian distribution since these methods does not depend on any specific distributions or any particular delay models.

7 References

[1] Fang, J., Gupta, S., Kumar, S. V., *et al.*: 'Circuit reliability: from physics to architectures'. Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD), San Jose, CA, USA, 2012

[2] Alam, M., Kang, K., Paul, B. C., *et al.*: 'Reliability and process variation aware design of VLSI design'. Proc. 14th Int.Symp. on the Physical and Failure Analysis of Integrated Circuits (IPFA), India, 2007

[3] Borkar, S., Karnik, T., Narendra, S., *et al.*: 'Parameter variation and impact on circuits and microarchitecture'. Proc. 40th Design Autom. Conf. (DAC), Anaheim, CA, USA, 2003

[4] Forzan, D.P.C.: 'Statistical static timing analysis: a survey', *Integr. VLSI J.*, 2008, **42**, (3), pp. 409–435

[5] Blaauw, D., Chopra, K., Srivastava, A., *et al.*: 'Statistical timing analysis: from basic principles to state of the art', *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2008, **27**, (4), pp. 589–607

[6] Visweswariah, C., Ravindran, K., Kalafala, K., *et al.*: 'First-order incremental block-based statistical timing analysis', *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2006, **25**, (10), pp. 2170–2180

[7] Agarwal, A., Blaauw, D., Zolotov, V.: 'Statistical timing analysis for intra-die process variations with spatial correlations'. Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD), San Jose, CA, USA, 2003

[8] Zhang, L., Chen, W., Hu, Y., *et al.*: 'Correlation-preserved statistical timing with a quadratic form of Gaussian variables', *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2006, **25**, (11), pp. 2437–2449

[9] Chang, H., Sapatnekar, S.S.: 'Statistical timing analysis under spatial correlations', *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2005, **24**, (9), pp. 1467–1482

[10] Feng, Z., Li, P., Zhan, Y.: 'Fast second-order statistical static timing analysis using parameter dimension reduction'. Proc. 44th Design Autom. Conf. (DAC), San Jose, CA, USA, 2007

[11] Yu, Y., Jha, N. K.: 'Statistical optimization of FinFET processor architectures under PVT variations using dual device-type assignment', *J. Emerg. Technol. Comput. Syst.*, 2017, **14**(1), Article 3, 25 pages

[12] Hwang, E.J., Kim, W., Kim, Y.H.: 'Timing yield slack for timing yield-constrained optimization and its application to statistical leakage minimization', *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2013, **21**, (10), pp. 1783–1796

[13] Agarwal, A., Chopra, K., Blaauw, D., *et al.*: 'Circuit optimization using statistical static timing analysis'. Proc. 42nd Design Autom. Conf. (DAC), Anaheim, CA, USA, 2005

[14] Bhardwaj, S., Vrudhula, S. B. K.: 'Leakage minimization of nano-scale circuits the presence of systematic and random variations'. Proc. 42nd Design Autom. Conf. (DAC), Anaheim, CA, USA, 2005

[15] Mani, M., Devgan, A., Orshansky, M.: 'An efficient algorithm for statistical minimization of total leakage power under timing yield constraints'. Proc. 42nd Design Autom. Conf. (DAC), Anaheim, CA, USA, 2005

[16] Sinha, D., Shenoy, N. V., Zhou, H.: 'Statistical gate sizing for timing yield optimization'. Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD), San Jose, CA, USA, 2006

[17] Agarwal, A., Chopra, K., Blaauw, D.: 'Statistical timing based optimization using gate sizing'. Proc. Conf. Design, Automation Test Europe (DATE), Munich, Germany, 2005

[18] Davoodi, A., Srivastava, A.: 'Variability driven gate sizing for binning yield optimization', *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, 2008, **16**, (6), pp. 683–692

[19] Park, H.S., Kim, W., Hyun, D.J., *et al.*: 'Timing criticality for timing yield optimization', *IEICE Trans. Fundam. Electron. Commun.Comput. Sci.*, 2008, **E91-A**, (12), pp. 3497–3505

[20] Chopra, K., Shah, S., Srivastava, A., *et al.*: 'Parametric yield maximization using gate sizing based on efficient statistical power and delay gradient computation'. Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD), San Jose, CA, USA, 2005

[21] Xiong, J., Zolotov, V., Visweswariah, C.: 'Incremental criticality and yield gradients'. Proc. Conf. Design, Automation Test Europe (DATE), Munich, Germany, 2008

[22] Srivastava, A., Sylvester, D., Blaauw, D.: 'Statistical optimization of leakage power considering process variations using dual-Vth and sizing'. Proc. 41st Design Autom. Conf. (DAC), San Diego, CA, USA, 2004

[23] Beece, D. K., Xiong, J., Visweswariah, C., *et al.*: 'Transistor sizing of custom high performance digital circuits with parametric yield considerations'. Proc. 47th Design Autom. Conf. (DAC), Anaheim, CA, USA, 2010

[24] Li, X., Le, J., Celik, M., *et al.*: 'Defining statistical sensitivity for timing optimization of logic circuits with large-scale process and environmental variations'. Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD), San Jose, CA, 2005

[25] Zhan, Y., Strojwas, A. J., Sharma, M., *et al.*: 'Statistical critical path analysis considering correlations'. Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD), SAN Jose, CA, USA, 2005

[26] Xiong, J., Zolotov, V., Visweswariah, C., *et al.*: 'Criticality computation parameterized statistical timing'. Proc. Design Autom. Conf. (DAC), San Francisco, CA, USA, 2006

[27] Chung, J., Abraham, J.A.: 'On computing criticality in refactored timing graphs', *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2012, **31**, (12), pp. 1935–1939

[28] Ramprasath, S., Vasudevan, V.: 'Statistical criticality computation using the circuit delay', *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2014, **33**, (5), pp. 717–727

[29] Ramprasath, S., Vasudevan, V.: 'Efficient algorithms for discrete gate sizing and threshold voltage assignment based on an accurate analytical statistical yield gradient', *ACM Trans. Des. Autom. Electron. Syst.*, 2016, **21**, (4),p.66

[30] Coudert, O.: 'Gate sizing for constrained delay/power/area optimization', *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, 1997, **5**, (4), pp. 465–472

[31] Xiang, H., Reddy, L., Qian, H., *et al.*: 'Gate movement for timing improvement on row based dual-VDD designs'. Proc. (17th) Int. Symp. Quality Electron. Design (ISQED), Santa Clara, CA, USA, 2016, pp. 423–429

[32] Xiang, H., Qian, H., Zhou, C., *et al.*: 'Row based dual-VDD island generation and placement'. Proc. 51st Design Autom. Conf. (DAC), San Francisco, CA, USA, 2014, pp. 1–6

[33] Sreenath, K., Ramesh, S.R.: 'Statistical viability analysis and optimization through gate sizing', in '*Advanced computational and communication paradigms*' (Springer, Singapore, 2018), pp. 149–155

[34] Li, B., Schlichtmann, U.: 'Statistical timing analysis and criticality computation for circuits with post-silicon clock tuning elements', *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2015, **34**, (11), pp. 1784–1797

[35] Perez, Z., Villacorta, H., Champac, V.: 'An accurate novel gate-sizing metric to optimize circuit performance under local intra-die process variations'. IFIP/IEEE Int.Conf. on Very Large Scale Integration (VLSI-SoC), Verona, Italy, 2018, pp. 77–82

[36] Sinha, D., Shenoy, N.V., Zhou, H.: 'Statistical timing yield optimization by gate sizing', *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, 2006, **14**, (10), pp. 1140–1146

[37] Wang, F., Xie, Y., Ju, H.: 'A novel criticality computation method statistical timing analysis'. Proc. Conf. Design, Automation Test Europe (DATE), Nice, France, 2007, pp. 1611–1616

© The Institution of Engineering and Technology 2019

17518598, 2019, 7, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-cds.2018.5616 by Thapar University, Wiley Online Library on [23/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License